

Design - Revision 2

1. Storage Model

Use SQLite, not JSON, for Phase 1.

Core Tables

```
tasks(  
  
    id TEXT PRIMARY KEY,  
  
    title TEXT,  
  
    owner_session TEXT,  
  
    notify_target TEXT,  
  
    observable_type TEXT,  
  
    observable_id TEXT,  
  
    checkin_every_minutes INTEGER,  
  
    started_at TEXT,  
  
    last_observed_at TEXT,  
  
    last_observed_status TEXT,
```

```
observation_source TEXT,  
  
last_notification_sent_at TEXT,  
  
last_notification_status TEXT,  
  
resolution_status TEXT,  
  
resolution_at TEXT  
  
)
```

```
task_events(  
  
    id INTEGER PRIMARY KEY,  
  
    task_id TEXT,  
  
    event_type TEXT,  
  
    observed_status TEXT,  
  
    message TEXT,  
  
    created_at TEXT  
  
)
```

2. Concurrency Model

- Single SQLite database file under a dedicated state path, not under memory/.
- All writes are transactional.
- Watchdog and registration logic use short explicit transactions.
- No multi-process shared JSON mutation.

3. State Definitions

- **running**: active source confirms task is still live
- **completed**: terminal success signal observed
- **failed**: terminal failure signal observed
- **stale**: still running but overdue for operator update
- **unknown**: no longer confidently observable, no terminal outcome proven

4. Notification Logic

- completion -> send immediately once
- failure -> send immediately once
- stale/running -> send only when interval elapsed and notification state changed or is due
- unknown -> send degraded-status notice when unknown threshold is crossed

5. Design Constraint

Phase 1 notification and evaluation logic shall be deterministic and template-based. No routine LLM invocation is required or assumed.